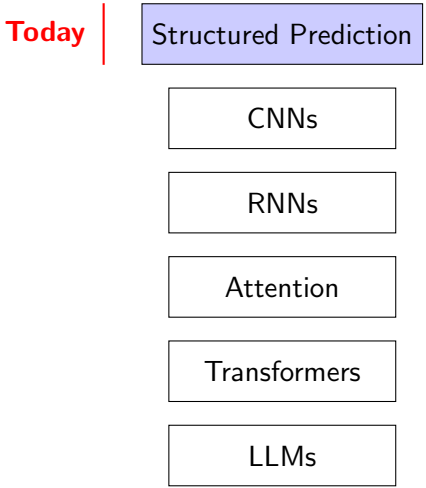
Structured Prediction for Sequences

CS 115B: Foundations of NLP II

James Pustejovsky

February 13, 2026

Course Roadmap



What is Structured Prediction?

Classification: Predict a single label

- Input: document $\mathbf{x}$
- Output: sentiment $y \in \{\text{pos}, \text{neg}\}$

Structured Prediction: Predict a structured output

- Input: sequence $\mathbf{x} = (x_1, \ldots, x_n)$
- Output: sequence $\mathbf{y} = (y_1, \ldots, y_n)$
- *Structure*: outputs have dependencies

Key Challenge

Output space is exponentially large: $|\mathcal{Y}| = K^n$ for K labels and length n

Sequence Labeling Tasks in NLP

1. **Part-of-Speech (POS) Tagging**

The	cat	sat	on	the	mat
DET	NOUN	VERB	ADP	DET	NOUN

2. **Named Entity Recognition (NER)**

Apple	hired	Tim	Cook	in	California	.
B-ORG	O	B-PER	I-PER	O	B-LOC	O

3. **Chunking, Word Segmentation, Lemmatization, ...**

Common pattern: Local decisions (per-token labels) with global constraints

Why Not Independent Classification?

Naive approach: Treat each position independently

$$y_i = \arg\max_{y' \in \mathcal{Y}} P(y' \mid x_i)$$

Problems:

1. Ignores label dependencies
   - In English: DET usually followed by NOUN/ADJ, not VERB
   - In NER: I-PER must follow B-PER or I-PER
2. No global consistency
   - Might predict B-PER, O, I-PER (invalid!)
3. Misses context from neighboring decisions

Need: Models that capture output structure

Notation and Setup

Input: $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ where $x_i \in \mathcal{X}$ (e.g., words)
Output: $\mathbf{y} = (y_1, y_2, \ldots, y_n)$ where $y_i \in \mathcal{Y}$ (e.g., tags)

Goal: Learn $f : \mathcal{X}^* \to \mathcal{Y}^*$

Training data: $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{N}$

Key modeling choices:

1. How to score candidate outputs $\mathbf{y}$ given input $\mathbf{x}$?
2. How to find the best output (inference)?
3. How to learn model parameters (training)?

Outline: Traditional Sequence Models

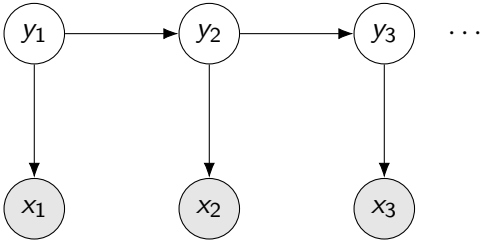
1. **Hidden Markov Models (HMMs)**
   - Generative model
   - Strong independence assumptions
2. **Maximum Entropy Markov Models (MEMMs)**
   - Discriminative, locally normalized
   - Label bias problem
3. **Conditional Random Fields (CRFs)**
   - Discriminative, globally normalized
   - State-of-the-art linear model
4. **Structured Perceptron**
   - Simple online learning
   - No probabilistic assumptions

Hidden Markov Models (HMMs)

Generative model: $P(\mathbf{x}, \mathbf{y})$

Factorization:

$$P(\mathbf{x}, \mathbf{y}) = \prod_{i=1}^{n} P(y_i \mid y_{i-1}) \cdot P(x_i \mid y_i)$$



Parameters:

- Transition: $P(y_i \mid y_{i-1})$ — tag bigram probabilities
- Emission: $P(x_i \mid y_i)$ — word given tag

HMM: Inference and Training

Inference: Find best tag sequence

$$\mathbf{y}^* = \arg\max_{\mathbf{y}} P(\mathbf{y} \mid \mathbf{x}) = \arg\max_{\mathbf{y}} P(\mathbf{x}, \mathbf{y})$$

Viterbi algorithm: Dynamic programming, $O(nK^2)$

Training: Maximum likelihood

- Count occurrences in labeled data
- Normalize to get probabilities
- Simple but requires labeled data

Limitations:

- Strong independence assumptions
- Hard to incorporate rich features
- Generative model (models $P(\mathbf{x})$ unnecessarily)

Maximum Entropy Markov Models (MEMMs)

Discriminative model: $P(\mathbf{y} \mid \mathbf{x})$

Local normalization:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^{n} P(y_i \mid y_{i-1}, \mathbf{x})$$

Each $P(y_i \mid y_{i-1}, \mathbf{x})$ is a softmax (MaxEnt) classifier:

$$P(y_i \mid y_{i-1}, \mathbf{x}) = \frac{\exp(\mathbf{w}^\top \mathbf{f}(y_{i-1}, y_i, \mathbf{x}, i))}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{w}^\top \mathbf{f}(y_{i-1}, y', \mathbf{x}, i))}$$

Advantages:

- Can use rich features $\mathbf{f}$ (e.g., word shape, prefixes, context)
- Discriminative

Problem: Label bias — local normalization prefers states with fewer outgoing transitions

Conditional Random Fields (CRFs)

Global normalization:

$$P(\mathbf{y} \mid \mathbf{x}; \mathbf{w}) = \frac{\exp(\mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}))}{\sum_{\mathbf{y}' \in \mathcal{Y}^n} \exp(\mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}'))}$$

where $\Phi(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbf{f}(y_{i-1}, y_i, \mathbf{x}, i)$ (feature sum)

Key insight: Score entire sequence, normalize over all possible sequences

Advantages:

- No label bias
- Rich features
- Theoretically principled

Challenge: Computing partition function $Z(\mathbf{x}) = \sum_{\mathbf{y}'} \exp(\mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}'))$ requires forward-backward algorithm

Comparison: HMM vs MEMM vs CRF

	HMM	MEMM	CRF
Model	Generative	Discriminative	Discriminative
Normalization	Per-state	Local	Global
Features	Limited	Rich	Rich
Label bias	No	Yes	No
Training	Counting	MaxEnt	Gradient descent
Inference	Viterbi	Viterbi	Viterbi

CRFs were state-of-the-art linear models for sequences (2001-2015)

Today: Neural models dominate, but understanding CRFs helps understand neural+CRF hybrids

The Viterbi Algorithm: Motivation

Problem: Find best sequence

$$\mathbf{y}^* = \arg\max_{\mathbf{y} \in \mathcal{Y}^n} \text{score}(\mathbf{x}, \mathbf{y})$$

Naive search: $O(K^n)$ — exponential!

Key observation: Scoring decomposes

$$\text{score}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} s(y_{i-1}, y_i, \mathbf{x}, i)$$

where $s(y_{i-1}, y_i, \mathbf{x}, i)$ is a *local score* (transition + emission)

Dynamic programming: Optimal substructure

- Best path to position i with tag y_i uses best path to $i-1$
- Store best scores in table: $O(nK^2)$ — polynomial!

Viterbi Algorithm: Formulation

Define $\delta_i(y)$ = score of best path ending at position i with tag y

Recursion:

$$\delta_i(y) = \max_{y' \in \mathcal{Y}} [\delta_{i-1}(y') + s(y', y, \mathbf{x}, i)]$$

Base case: $\delta_1(y) = s(\text{START}, y, \mathbf{x}, 1)$

Backpointers: Store argmax to reconstruct path

$$\psi_i(y) = \arg\max_{y' \in \mathcal{Y}} [\delta_{i-1}(y') + s(y', y, \mathbf{x}, i)]$$

Final answer:

$$\mathbf{y}^* = \text{backtrack from } \arg\max_{y \in \mathcal{Y}} \delta_n(y)$$

Viterbi Algorithm: Pseudocode

Require: Sequence $\mathbf{x}$, scoring function s, tag set $\mathcal{Y}$

Initialize $\delta_1(y) = s(\text{START}, y, \mathbf{x}, 1)$ for all $y \in \mathcal{Y}$
for $i = 2$ to n **do**
  for each $y \in \mathcal{Y}$ **do**
    $\delta_i(y) = \max_{y' \in \mathcal{Y}} [\delta_{i-1}(y') + s(y', y, \mathbf{x}, i)]$
    $\psi_i(y) = \arg\max_{y' \in \mathcal{Y}} [\delta_{i-1}(y') + s(y', y, \mathbf{x}, i)]$
  end for
end for
$y_n^* = \arg\max_{y \in \mathcal{Y}} \delta_n(y)$
for $i = n - 1$ down to 1 **do**
  $y_i^* = \psi_{i+1}(y_{i+1}^*)$
end for
return $\mathbf{y}^* = (y_1^*, \ldots, y_n^*)$

Complexity: $O(nK^2)$ time, $O(nK)$ space

Viterbi Example: POS Tagging

Sentence: "The cat sat"
Tags: $\mathcal{Y} = \{\text{DET, NOUN, VERB}\}$

Scores (simplified, log probabilities):

	DET	NOUN	VERB
"The"	-0.1	-4.0	-5.0
"cat"	-5.0	-0.2	-4.0
"sat"	-6.0	-3.0	-0.3

$y_{i-1} \backslash y_i$	DET	NOUN	VERB
START	-0.5	-3.0	-4.0
DET	-5.0	-0.3	-3.0
NOUN	-4.0	-2.0	-0.5
VERB	-3.0	-2.0	-4.0

Position 1 ("The"):

- $\delta_1(\text{DET}) = -0.5 + (-0.1) = -0.6$
- $\delta_1(\text{NOUN}) = -3.0 + (-4.0) = -7.0$
- $\delta_1(\text{VERB}) = -4.0 + (-5.0) = -9.0$

Viterbi Example: Continued

Position 2 ("cat"):

$$\delta_2(\text{NOUN}) = \max \begin{cases} \delta_1(\text{DET}) + s(\text{DET}, \text{NOUN}) + s(\text{NOUN}, \text{cat}) \\ \delta_1(\text{NOUN}) + s(\text{NOUN}, \text{NOUN}) + \cdots \\ \delta_1(\text{VERB}) + s(\text{VERB}, \text{NOUN}) + \cdots \end{cases}$$
$$= \max\{-0.6 - 0.3 - 0.2, \ldots\} = -1.1$$

Continue for all tags at position 2, then position 3...

Key insight: At each position, only track best path to each tag (not all K^i paths!)

From Binary to Structured Perceptron

Binary Perceptron (Rosenblatt 1958):

- Linear classifier: $y = \text{sign}(\mathbf{w}^\top \mathbf{x})$
- Update rule: If $y \neq \hat{y}$, then $\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$

Structured Perceptron (Collins 2002):

- Score entire structures: $\text{score}(\mathbf{x}, \mathbf{y}) = \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y})$
- Prediction: $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}'} \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}')$
- Update: If $\mathbf{y} \neq \hat{\mathbf{y}}$, then

$$\mathbf{w} \leftarrow \mathbf{w} + \Phi(\mathbf{x}, \mathbf{y}) - \Phi(\mathbf{x}, \hat{\mathbf{y}})$$

Intuition: Increase score of correct structure, decrease score of predicted structure

Structured Perceptron: Formal Algorithm

Algorithm 1 Structured Perceptron

Initialize $\mathbf{w} = \mathbf{0}$
for $t = 1$ to T (epochs) **do**
  for each $(\mathbf{x}, \mathbf{y})$ in training data **do**
    Predict: $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}' \in \mathcal{Y}^n} \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}')$ *// Viterbi*
    if $\hat{\mathbf{y}} \neq \mathbf{y}$ **then**
      Update: $\mathbf{w} \leftarrow \mathbf{w} + \Phi(\mathbf{x}, \mathbf{y}) - \Phi(\mathbf{x}, \hat{\mathbf{y}})$
    end if
  end for
end for
return $\mathbf{w}$

Key components:

1. Feature function $\Phi(\mathbf{x}, \mathbf{y})$
2. Inference algorithm (Viterbi for sequences)
3. Simple online learning (no probabilistic training)

Why Does Structured Perceptron Work?

Theorem (Collins 2002): If data is separable with margin γ, structured perceptron makes at most R^2/γ^2 mistakes, where $R = \max_{\mathbf{x},\mathbf{y}} ||\Phi(\mathbf{x}, \mathbf{y})||$

Intuition: Perceptron moves towards correct structure

- Increases dot product $\mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y})$ for gold structures
- Decreases dot product for incorrect predictions
- Eventually separates correct from incorrect (if separable)

Advantages:

- Simple to implement
- No need for partition function (unlike CRF)
- Fast training
- Often competitive accuracy

Averaged Perceptron

Problem: Final weights depend on order of examples

Solution (Freund & Schapire 1999): Average all weight vectors

$$\mathbf{w}_{\text{avg}} = \frac{1}{NT} \sum_{t=1}^{T} \sum_{i=1}^{N} \mathbf{w}^{(t,i)}$$

Efficient implementation: Track cumulative sum

- Maintain $\mathbf{w}$ (current) and $\mathbf{u}$ (sum)
- After each update: $\mathbf{u} \leftarrow \mathbf{u} + \mathbf{w}$
- Return $\mathbf{w}_{\text{avg}} = \mathbf{u}/(\text{total updates})$

Impact: Significant accuracy improvements (typically 1-3%)

Feature Functions for Sequences

Recall: $\Phi(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbf{f}(y_{i-1}, y_i, \mathbf{x}, i)$

Each $\mathbf{f}$ is a feature vector that can look at:

- Current word x_i and tags y_{i-1}, y_i
- Context: x_{i-1}, x_{i+1}, etc.
- Word properties: capitalization, prefixes, suffixes

Example features for POS tagging:

$$f_1 : \mathbb{I}[y_{i-1} = \text{DET}, y_i = \text{NOUN}]$$
$$f_2 : \mathbb{I}[x_i = \text{"cat"}, y_i = \text{NOUN}]$$
$$f_3 : \mathbb{I}[x_i \text{ ends in "ing"}, y_i = \text{VERB}]$$
$$f_4 : \mathbb{I}[x_i \text{ is capitalized}, y_i = \text{PROPN}]$$

Feature Templates

Instead of hand-crafting each feature, use **feature templates**:

Template	**Example Instantiation**
$\langle y_i, x_i \rangle$	$\langle \text{NOUN}, \text{"cat"} \rangle$
$\langle y_{i-1}, y_i \rangle$	$\langle \text{DET}, \text{NOUN} \rangle$
$\langle y_i, x_{i-1} \rangle$	$\langle \text{VERB}, \text{"cat"} \rangle$
$\langle y_i, x_{i+1} \rangle$	$\langle \text{NOUN}, \text{"sat"} \rangle$
$\langle y_i, \text{prefix}(x_i, 2) \rangle$	$\langle \text{VERB}, \text{"un"} \rangle$
$\langle y_i, \text{suffix}(x_i, 3) \rangle$	$\langle \text{VERB}, \text{"ing"} \rangle$
$\langle y_i, \text{contains-hyphen}(x_i) \rangle$	$\langle \text{ADJ}, \text{True} \rangle$
$\langle y_i, \text{shape}(x_i) \rangle$	$\langle \text{PROPN}, \text{"Xxxxx"} \rangle$

Automatic feature generation: Instantiate templates on training data

Feature Engineering: Best Practices

Good features:

1. **Expressive**: Capture relevant patterns
2. **Generalizable**: Not overfitted to training data
3. **Efficient**: Not too many (millions OK, not billions)

Common feature types:

- Lexical: exact word, lowercase word, lemma
- Orthographic: capitalization, contains-digit, punctuation
- Morphological: prefix/suffix up to length 3-4
- Contextual: words in window $[i-2, i+2]$
- Shape: "Xxxxx" (capital), "xxxx" (lowercase), "dd" (digits)

Feature conjunctions: Combine base features

$$\langle y_i, x_i, x_{i+1} \rangle, \quad \langle y_{i-1}, y_i, x_i \rangle$$

Example: NER Feature Engineering

Sentence: "Steve Jobs founded Apple in California"

Features for x_i = "Apple", y_i = B-ORG:

- $\langle$B-ORG, "Apple"$\rangle$
- $\langle$B-ORG, "founded"$\rangle$ (previous word)
- $\langle$B-ORG, "in"$\rangle$ (next word)
- $\langle$B-ORG, shape=Xxxxx$\rangle$ (capitalized)
- $\langle$I-PER, B-ORG$\rangle$ (transition from "Jobs")
- $\langle$B-ORG, prefix="App"$\rangle$
- $\langle$B-ORG, in-gazetteer=company$\rangle$ (external knowledge)

Each fired feature contributes to score via its weight w_j

Implementation: Feature Vectors

Challenge: Feature vectors can be very high-dimensional (millions)

Sparse representation:

- Most features are 0 for any given $(\mathbf{x}, \mathbf{y})$ pair
- Store only non-zero indices and values
- Use hash maps: feature name $\rightarrow$ weight

Example (Python-like pseudocode):

```
features = {}
features[('tag', y_i, 'word', x_i)] = 1
features[('trans', y_i-1, y_i)] = 1
features[('tag', y_i, 'suffix', x_i[-3:])] = 1
score = sum(weights.get(f, 0) for f in features)
```

Structured Perceptron: Complete Training

Algorithm 2 Averaged Structured Perceptron

Initialize $\mathbf{w} = \mathbf{0}$, $\mathbf{u} = \mathbf{0}$, $c = 0$
for $t = 1$ to T **do**
  for each $(\mathbf{x}, \mathbf{y})$ in shuffled training data **do**
    $\hat{\mathbf{y}} = \text{Viterbi}(\mathbf{x}, \mathbf{w})$
    if $\hat{\mathbf{y}} \neq \mathbf{y}$ **then**
      $\mathbf{w} \leftarrow \mathbf{w} + \Phi(\mathbf{x}, \mathbf{y}) - \Phi(\mathbf{x}, \hat{\mathbf{y}})$
    end if
    $\mathbf{u} \leftarrow \mathbf{u} + \mathbf{w}$, $c \leftarrow c + 1$
  end for
end for
return $\mathbf{w}_{\text{avg}} = \mathbf{u}/c$

Hyperparameters: Number of epochs T (typically 5-20)

Early Stopping and Validation

Problem: When to stop training?

Solution: Monitor performance on validation set

1. Split data: train / dev / test
2. After each epoch:
   - Run Viterbi on dev set
   - Compute accuracy
3. Keep weights from epoch with best dev accuracy
4. Report final results on test set

Evaluation metrics:

- Per-token accuracy: $\frac{\#\text{ correct tags}}{\#\text{ total tokens}}$
- Per-sequence accuracy: % of sentences completely correct
- F1-score: For NER, compute precision/recall of entities

Inference: Constraint Incorporation

Viterbi finds: $\arg\max_{\mathbf{y}} \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y})$

Can easily incorporate **hard constraints**:

Example (NER): "I-PER must follow B-PER or I-PER"

- During Viterbi, set $s(y', y) = -\infty$ for invalid transitions
- Ensures output respects BIO constraints

Example (POS): "DET cannot follow DET"

- Encode linguistic constraints into transition scores

Advantage over neural softmax: Guaranteed valid outputs

Beam Search: Approximate Inference

Problem: For some tasks, exact inference is intractable

Beam search: Keep top-k hypotheses at each step

1. Initialize: beam $= \{(\epsilon, 0)\}$ (empty sequence, score 0)
2. For position $i = 1$ to n:
   - For each hypothesis $(\mathbf{y}_{1:i-1}, s)$ in beam:
   - Expand with all possible tags $y_i \in \mathcal{Y}$
   - Compute scores for all expansions
   - Keep top-k by total score
3. Return best complete hypothesis

Complexity: $O(nkK)$ instead of $O(nK^2)$
Trade-off: Speed vs. optimality guarantee

Loss-Augmented Training

Standard perceptron: Distinguish gold from predicted

Problem: All errors treated equally

Loss-augmented inference:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}'} [\mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}') + \Delta(\mathbf{y}, \mathbf{y}')]$$

where $\Delta(\mathbf{y}, \mathbf{y}')$ is a loss function (e.g., Hamming distance)

Effect: Find most problematic incorrect prediction (violates margin most)

Connection to structured SVM: Max-margin training

Structured SVM (Max-Margin Markov Networks)

Objective (structural SVM):

$$\min_{\mathbf{w}} \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{N}\xi_i$$

subject to:

$$\mathbf{w}^\top[\Phi(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) - \Phi(\mathbf{x}^{(i)}, \mathbf{y}')] \geq \Delta(\mathbf{y}^{(i)}, \mathbf{y}') - \xi_i$$

for all i and all $\mathbf{y}' \neq \mathbf{y}^{(i)}$

Interpretation: Gold structure should score higher than any other by at least the loss

Training: Cutting plane algorithm (iterate: find most violated constraint, add to working set)

Advantage over perceptron: Regularization, margin-based

Higher-Order Models

So far: First-order (bigram) transitions

$$\text{score}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} s(y_{i-1}, y_i, \mathbf{x}, i)$$

Higher-order: Trigram, skip-chain, etc.

$$\text{score}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} s(y_{i-2}, y_{i-1}, y_i, \mathbf{x}, i)$$

Trade-offs:

- More expressive (capture longer dependencies)
- Slower inference: $O(nK^3)$ for trigrams
- Sparser features (data fragmentation)

Practical: Second-order often good balance

Semi-Markov Models

Standard sequence labeling: One label per token

Semi-Markov CRF: Segment-based labeling

- Output: sequence of segments with labels
- Example (NER): [Steve Jobs]$_{\text{PER}}$ [founded]$_{O}$ [Apple]$_{\text{ORG}}$

Advantages:

- Features on entire segments (not just tokens)
- Natural for chunking tasks
- Can capture segment-level properties

Inference: Dynamic programming over segments ($O(nL^2K^2)$ where L is max segment length)

From Linear to Neural Models

Linear structured models:

$$\text{score}(\mathbf{x}, \mathbf{y}) = \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y})$$

Features Φ are hand-crafted, sparse, discrete

Neural structured models:

$$\text{score}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbf{w}^\top f_\theta(x_i, x_{i-1}, \ldots) + s(y_{i-1}, y_i)$$

where f_θ is a neural network (LSTM, Transformer)

Neural+CRF (BiLSTM-CRF):

- BiLSTM computes contextual representations
- CRF layer models transitions
- Train end-to-end with backprop

This bridges to your next lectures on RNNs and attention!

Implementation Tips

1. **Feature hashing**: Map feature strings to indices
   - Use hash function to map to fixed-size array
   - Trade-off: collisions vs. memory
2. **Sparse updates**: Only update weights for active features
   - Feature vectors are very sparse
   - Use dict/hashmap, not dense arrays
3. **Efficient Viterbi**: Vectorize operations
   - Compute scores for all tags in parallel
   - Use NumPy broadcasting
4. **Feature pruning**: Remove rare features
   - Features seen $< k$ times in training
   - Reduces memory, speeds inference

Common Pitfalls

1. **Not shuffling data**: Perceptron sensitive to order
2. **Forgetting to average**: Significant accuracy loss
3. **Too many features**: Overfitting, slow inference
   - Use feature count cutoffs
   - Regularization (L2 penalty)
4. **Ignoring constraints**: Invalid BIO sequences
5. **Not lowercasing**: "The" vs "the" treated differently
   - Consider both cased and uncased features
6. **Off-by-one errors**: In Viterbi backtracking!

Software and Libraries

Classical structured prediction:

- **CRFsuite**: Fast CRF implementation in C
- **sklearn-crfsuite**: Python wrapper
- **python-crfsuite**: Another Python interface
- **Wapiti**: CRF toolkit with various training algorithms

Neural structured prediction:

- **AllenNLP**: High-level NLP library (PyTorch)
- **Flair**: Sequence labeling library
- **Hugging Face**: Transformers + token classification

Datasets:

- CoNLL 2003 (NER), Penn Treebank (POS), OntoNotes

Historical Context and Impact

Timeline:

- 1989: HMMs for POS tagging (Church)
- 2000: MEMMs (McCallum et al.)
- 2001: CRFs (Lafferty, McCallum, Pereira)
- 2002: Structured perceptron (Collins)
- 2004-2014: CRFs dominate sequence labeling
- 2015+: Neural models (BiLSTM-CRF, Transformers)

Impact:

- Established discriminative training for NLP
- Feature engineering as art form
- Bridged ML and linguistics
- Foundation for modern neural architectures

Performance Comparison

Typical accuracy on POS tagging (Penn Treebank):

Model	Accuracy
Baseline (most frequent tag)	90%
HMM	95%
MEMM	96%
CRF	97%
Structured Perceptron	96.5%
BiLSTM	97.5%
BiLSTM-CRF	97.8%
BERT + CRF	98.5%

Numbers approximate, vary by exact setup

Connection to Neural Networks

Structured perceptron teaches key concepts for neural NLP:

1. **Structured outputs**: Neural models also predict sequences
   - Seq2seq, transformers output sequences
2. **Scoring functions**: Neural nets learn score($\mathbf{x}, \mathbf{y}$)
   - Features Φ replaced by learned representations
3. **Inference**: Still need decoding algorithms
   - Greedy, beam search, Viterbi
4. **Training**: Same principle (increase gold score, decrease incorrect)
   - Backprop = gradient-based perceptron update

Next: Neural Sequence Models

Coming up in the course:

- **CNNs**: Local feature detection, character-level models
- **RNNs/LSTMs**: Recurrent models for sequences
  - Replace hand-crafted features with learned representations
  - BiLSTM-CRF = neural + structured
- **Attention**: Weighted context aggregation
- **Transformers**: Self-attention for sequences
  - BERT, GPT for sequence labeling
  - No explicit structure modeling (usually)
- **LLMs**: Few-shot and zero-shot sequence labeling

Structured prediction provides foundation for understanding these models!

Key Takeaways

1. **Structured prediction** models dependencies in outputs
2. **Viterbi algorithm**: Efficient DP for sequence inference
3. **Feature engineering**: Crucial for linear models
4. **Structured perceptron**: Simple, effective learning algorithm
   - Online updates: $\mathbf{w} \leftarrow \mathbf{w} + \Phi(\mathbf{x}, \mathbf{y}) - \Phi(\mathbf{x}, \hat{\mathbf{y}})$
   - Don't forget to average!
5. **Trade-offs**: CRF vs Perceptron vs Neural
   - CRF: principled, probabilistic
   - Perceptron: simple, fast
   - Neural: automatic features, SOTA
6. **Foundation for neural NLP**: Concepts carry forward

Recommended Reading

Foundational papers:

- Collins (2002): "Discriminative Training Methods for HMMs"
- Lafferty et al. (2001): "Conditional Random Fields"
- Taskar et al. (2003): "Max-Margin Markov Networks"

Textbooks:

- Jurafsky & Martin, *Speech and Language Processing*, Ch. 8
- Smith, *Linguistic Structure Prediction* (2011)
- Daumé III, *A Course in Machine Learning*, Ch. 11-12

Tutorials:

- Sutton & McCallum (2012): "An Introduction to CRFs"
- Collins lecture notes (available online)

Exercise: Implement Structured Perceptron

Mini-project (recommended):

Implement structured perceptron for POS tagging:

1. Load data (CoNLL format)
2. Implement feature extraction
3. Implement Viterbi algorithm
4. Implement perceptron training loop (with averaging)
5. Evaluate on dev/test sets

Extensions:

- Add more feature templates
- Implement beam search
- Compare to sklearn CRF
- Try on NER task

Great way to solidify understanding before neural models!

Questions?

Backup: Viterbi Algorithm Complexity

Analysis:

- Outer loop: n positions
- Inner loop: K tags (current)
- Max over: K tags (previous)
- Total: $O(nK^2)$

Space: $O(nK)$ for DP table + backpointers

Comparison to naive search:

- Naive: $O(K^n)$ — exponential
- Viterbi: $O(nK^2)$ — polynomial

Example: $n = 50$ tokens, $K = 45$ tags

- Naive: $45^{50} \approx 10^{82}$ (larger than atoms in universe!)
- Viterbi: $50 \times 45^2 = 101,250$ operations

Backup: Forward-Backward for Marginals

Viterbi: Finds most likely sequence (max)
Forward-Backward: Computes marginals (sum)

Forward $\alpha_i(y)$: Sum of scores of all paths ending at position i with tag y

$$\alpha_i(y) = \sum_{y' \in \mathcal{Y}} \alpha_{i-1}(y') \cdot \exp(s(y', y, \mathbf{x}, i))$$

Backward $\beta_i(y)$: Sum of scores of all paths from i to end

$$\beta_i(y) = \sum_{y' \in \mathcal{Y}} \beta_{i+1}(y') \cdot \exp(s(y, y', \mathbf{x}, i+1))$$

Marginal:

$$P(y_i = y \mid \mathbf{x}) = \frac{\alpha_i(y) \cdot \beta_i(y)}{Z(\mathbf{x})}$$

Used in CRF training (gradient computation)